

Lab 6 Practice: Bayes and Naive Bayes Classifiers

CS201L: Artificial Intelligence Laboratory
Indian Institute of Technology, Dharwad

12 - Feb - 2026

Problem Statement

Using the Date Fruit dataset (`Date_Fruit_Datasets.csv`) prepared in the previous lab, implement and compare the performance of the Bayes classifier and the Naive Bayes classifier. You are required to categorize the fruit samples into their respective classes based on image-extracted numerical features.

Dataset Properties

- Total Samples: 898
- Total Features: 34 numerical features
- Target Attribute: `Class` (multiclass)
- Data Splits: Use the `DateFruit_Train.csv` (60%), `DateFruit_Validation.csv` (20%), and `DateFruit_Test.csv` (20%).

Task 1: Implementation of Naive Bayes Classifier

Implement the Naive Bayes classifier using the `GaussianNB()` class from Scikit-learn. Train the model using the training feature and target vectors, and generate predictions for the validation and test datasets.

Sample Naive Bayes code:

```
1 from sklearn.naive_bayes import GaussianNB
2
3 # Step 1: Initialize the Gaussian Naive Bayes model
4 naiveBayes = GaussianNB()
5
6 # Step 2: Model Training (Parameter estimation)
7 # Parameter estimation or model training is performed using the
  ↳ fit function
8 naiveBayes.fit(X_train, y_train)
9
```

```

10 # Step 3: Predict class labels for test or validation data
11 # Use the predict function to generate class levels
12 y_test_pred = naiveBayes.predict(X_test)

```

Task 2: Implementation of Bayes Classifier

Implement the Bayes classifier based on the Bayes decision rule. The classifier assumes that the features follow a Gaussian distribution. Follow these algorithmic steps:

Algorithm

Training

1. Load training, validation, and test datasets. Separate the input feature vectors ($\mathbf{x}$) from the target class labels ($\mathbf{y}$).
2. Class Separation: Group the training feature vectors into their respective classes C_i (where $i = 1, 2, \dots, M$ & M is the no. of classes).
3. Parameter Estimation: For each class C_i , compute the following from the training data:
 - 3.1 Mean Vector ($\boldsymbol{\mu}_i$): The average of all feature vectors in class i .
 - 3.2 Covariance Matrix ($\boldsymbol{\Sigma}_i$): Representing the relationships between features within class i .
 - 3.3 Prior Probability ($P(C_i)$): Calculated as:

$$P(C_i) = \frac{\text{Number of samples in class } i}{\text{Total training samples}}$$

Testing

1. For each sample $\mathbf{x}$ in the test data,
 - 1.1 Likelihood Computation: find the likelihood $p(\mathbf{x}|C_i)$ using the multivariate normal distribution density function (`multivariate_normal.pdf` from `scipy.stats`). Ensure that you set `allow_singular=True` to handle potentially singular covariance matrices.
 - 1.2 Posterior Probability Calculation: Compute the posterior for each class:

$$P(\hat{\boldsymbol{\mu}}_i, \hat{\boldsymbol{\Sigma}}_i|\mathbf{x}) = \frac{p(\mathbf{x}|\hat{\boldsymbol{\mu}}_i, \hat{\boldsymbol{\Sigma}}_i)P(C_i)}{\sum_{j=1}^M p(\mathbf{x}|\hat{\boldsymbol{\mu}}_j, \hat{\boldsymbol{\Sigma}}_j)P(C_j)}$$

where $P(\hat{\boldsymbol{\mu}}_i, \hat{\boldsymbol{\Sigma}}_i|\mathbf{x})$ is the posterior probability, $p(\mathbf{x}|\hat{\boldsymbol{\mu}}_i, \hat{\boldsymbol{\Sigma}}_i)$ is the likelihood, $P(C_i)$ is the prior, and $\sum_{j=1}^M p(\mathbf{x}|\hat{\boldsymbol{\mu}}_j, \hat{\boldsymbol{\Sigma}}_j)P(C_j)$ is the total sum probability.

1.3 Decision Rule: Assign the class label C_i that yields the highest posterior probability, and add it to the class prediction vector $\hat{C}$ such that,

$$\hat{C} = \underset{i}{\operatorname{argmax}} P(\hat{\mu}_i, \hat{\Sigma}_i | \mathbf{x})$$

2. Return the predicted class vector $\hat{C}$.

You can also refer to the class slides for specific parts of the algorithm.

```
1 import numpy as np
2 from scipy.stats import multivariate_normal
3
4 # Hint 1: To calculate likelihood, use
5   ↳ multivariate_normal.pdf() function
6 likelihood = multivariate_normal.pdf(x, mean=class_means[cls],
7   ↳ cov=class_covs[cls], allow_singular=True)
8
9 # Hint 2: To predict class label for test or validation data
10   ↳ sample, use np.argmax() function on posterior
11   ↳ probabilities. It selects the index of the highest value of
12   ↳ the given posterior probability array.
13 np.argmax(posterior)
14
15 # Hint 3: To make the calculation faster, use numpy operations
16 ### 1. Compute the likelihood for all samples together instead
17   ↳ of one sample at a time
18 likelihood = multivariate_normal.pdf(np.array(X_test),
19   ↳ mean=class_means[cls], cov=class_covs[cls],
20   ↳ allow_singular=True)
21
22 ### 2. Compute the numerator term together for all samples
23 numerator = likelihood * class_priors[cls]
24
25 ### 3. Compute total probability for all samples (after
26   ↳ calculation, confirm the dimension and the correctness
27   ↳ total probability)
28 total_probabilities = np.sum(numerator, axis=1)
29
30 ### 4. Compute posteriors for all samples (after calculation,
31   ↳ confirm the dimension and the correctness posteriors)
32 posteriors = numerator/total_probabilities
33
34 ### 5. Get the predictions for all samples together (after
35   ↳ calculation, confirm the dimension and the correctness
36   ↳ y_test_pred)
37 y_test_pred = np.argmax(posterior, axis=1)
```

Task 3: Performance Metrics Evaluation

Evaluate the performance of both classifiers on the validation and test data. Report(print) the following metrics: Confusion Matrix, Accuracy, Precision, Recall, and F1-score (using micro for overall and macro-averaging for multiclass evaluation)